

Agent Harness Engineering: A Survey - 中文章节式精读版

说明：本文档是按原论文结构整理的中文精读版，不是逐字全文翻译。它保留原文的章节顺序、核心术语、图表编号和论证脉络，用中文转述每一章与主要小节的内容。原文：Junjie Li 等，*Agent Harness Engineering: A Survey*，OpenReview/PDF，2026。

论文基本信息

题目：Agent Harness Engineering: A Survey

作者：Junjie Li, Xi Xiao, Yunbei Zhang, Chen Liu, Lin Zhao, Xiaoying Liao, Yingrui Ji, Janet Wang, Jianyang Gu, Yingqiang Ge, Weijie Xu, Xi Fang, Xiang Xu, Tianchen Zhao, Youngeun Kim, Tianyang Wang, Jihun Hamm, Smita Krishnaswamy, Jun Huan, Chandan K. Reddy。

机构包括：Carnegie Mellon University, Yale University, Johns Hopkins University, Northeastern University, Tulane University, University of Alabama at Birmingham, The Ohio State University, Virginia Tech, Amazon。

原文来源：

- 项目页：<https://picrew.github.io/LLM-Harness/>
- PDF：<https://picrew.github.io/LLM-Harness/main.pdf>
- OpenReview：<https://openreview.net/forum?id=eONq7FdiHa>

摘要精读

论文的核心判断是：在生产环境里，LLM agent 的可靠性往往不只由模型能力决定，更受模型外层的执行基础设施影响。作者把这层基础设施称为 agent execution harness，也就是把模型调用、上下文、工具、执行环境、状态、评测、监控和治理连接起来的工程底座。

论文提出三条主张。第一，agent harness 应被视为独立系统层，因为很多基准提升来自 harness 调整，而不是模型权重更新。第二，作者提出 ETCLOVG 七层分类法：Execution, Tooling, Context, Lifecycle, Observability, Verification, Governance。第三，作者把 170+ 开源项目映射到这个分类上，观察生态覆盖密度、薄弱环节和生产实践中的设计原则。

1 Introduction

1.1 Harness over Model

论文反对一个常见默认假设：只要模型足够强，加上足够好的 prompt，agent 就会可靠。作者认为，长程任务中的 agent 表现是“模型 + harness”的闭环结果。最近一些实验显示，在不改模型的情况下，仅通过改工具格式、系统提示、中间件上下文注入、自验证 hook 或自动 harness 优化，就能显著提升 coding/terminal benchmark 成绩。

这形成论文的 binding-constraint thesis：对长程 agent 任务而言，真正限制可靠性的瓶颈常常是执行底座，而不是单独的模型能力。

1.2 Practitioner-Research Gap

生产团队已经在实践中认识到 harness 的重要性。OpenAI 把 harness engineering 描述为围绕 Codex agent 设计环境、约束、文档和反馈循环的工程 discipline；Anthropic 的工程文章也反复强调简单可检查架构、面向 agent 的工具接口、渐进式上下文披露、长程任务的持久交接物和可恢复执行环境。

研究界虽然分别研究了 memory、tool use、planning 和 safety，但还缺少一个统一语言来描述这些组件如何组合成可靠系统。本文试图补上这个词汇和分类缺口。

1.3 Scope and Contributions

本文研究对象不是模型能力本身，也不是普通 chatbot 或轻量 API wrapper，而是把模型变成长程、可控、可观察、可恢复 agent 的基础设施层。贡献包括：

- 概念贡献：把 harness 作为现实 agent 可靠性的关键约束。
- 分类贡献：提出 ETCLOVG 七层框架，并把 Observability 与 Governance 提升为一等层。
- 经验贡献：映射大量公开项目，指出生态密集区和薄弱区。

2 Background and Taxonomy

2.1 Agent 系统演进

作者把 2022-2026 的演进分成三段。2022-2023 年是 ReAct/AutoGPT/BabyAGI 时代，系统主要是单循环、prompt、简单工具表和任务队列。2023-2024 年，工具调用学习、多 agent 协作和 benchmark 逐渐成熟，例如 Gorilla、ToolLLM、CAMEL、ChatDev、MetaGPT、SWE-bench、WebArena 等。到 2025-2026 年，生产经验让大家意识到，可靠性问题已经从“模型会不会”转向“底座是否能让模型稳定执行”。

2.2 三个工程阶段

Prompt engineering 关注单次调用的输入文本；context engineering 关注每一步模型应该看到哪些信息；harness engineering 则把整个运行时作为工程对象，包括执行环境、工具、状态、上下文、编排、监控、评测和治理。三者不是替代关系，而是包含关系：harness engineering 包含 context engineering，context engineering 又包含 prompt engineering。

2.3 ETCLOVG 七层分类

ETCLOVG 是论文的主框架：

- E - Execution Environment and Sandbox：agent 动作在哪里执行，受什么沙箱限制。
- T - Tool Interface and Protocol：工具如何描述、发现、调用和返回。
- C - Context and Memory Management：模型在每一步能看到什么，信息如何跨轮次和跨会话保存。
- L - Lifecycle and Orchestration：任务如何跨模型调用、工具调用、失败、修订和交接推进。
- O - Observability and Operations：如何记录 trace、成本、失败、延迟和可靠性信号。
- V - Verification and Evaluation：如何把任务和 trace 变成评测、失败归因和回归反馈。
- G - Governance and Security：如何通过权限、身份、策略、审计和人工确认约束 agent 行为。

前四层是结构核心，后三层是控制平面。论文特别强调 O 和 G 不应该只是 lifecycle hook 的副作用，因为生产环境里它们有独立工具栈、团队职责和失败模式。

2.4-2.9 语料范围与项目映射

作者收集公开可见的 agent harness 项目，包括论文、GitHub 项目、公司工程博客、工具文档和 curated list。纳入条件是：公开可查、实现或规定了具体 harness 机制、能映射到至少一个 ETCLOVG 层。排除简单 demo、prompt 包、薄 API wrapper、静态数据集、没有 agent-facing 机制的通用基础设施。

作者承认这个语料偏向英文、GitHub 可见、开源和 coding agent 生态。总体观察是：Execution、Tooling、Lifecycle 和 Verification 覆盖最密；Context 常嵌在框架中；Observability 和 Governance 在开源中较薄，更多出现在商业平台或工程实践文章中。

3 Execution Environment and Sandbox (E)

3.1 范围与定义

执行环境是 agent 动作真正发生的地方。对 LLM agent 来说，执行环境通常与沙箱绑定，因为 agent 可能自主运行命令、写文件、安装包、访问网络或操作图形界面。

3.1.2 沙箱为何成为 agent 时代的核心

沙箱有三种作用。第一是安全：LLM 生成动作不可预测，且可能被 prompt injection 劫持。第二是可复现：长程任务和 benchmark 需要把环境重置到已知状态。第三是活性：如果每个风险动作都要求人工批准，agent 无法长时间自主执行；沙箱定义一个可自由行动但边界清晰的区域。

3.2 沙箱类别

作者把 agent 沙箱分为七类：

- 通用托管沙箱：如 Daytona、E2B、Modal、Northflank、OpenSandbox、Docker Sandboxes，强调 API 化、弹性和隔离强度。
- Computer-use 基础设施：如 Anthropic Computer Use、CUA、OSWorld，让 agent 通过屏幕、鼠标、键盘操作 GUI。
- 代码专用沙箱：如 Judge0、OpenAI Code Interpreter、langchain-sandbox，强调快速启动、高并发、代码执行和数据分析。
- 框架内置 runtime：如 OpenHands runtime、agent-infra sandbox、smolagents executors，方便但与框架强耦合。
- 浏览器评测环境：如 WebArena、VisualWebArena、BrowserGym、WorkArena，同时是沙箱和 benchmark harness。

- OS级权限沙箱：如 Anthropic sandbox-runtime、Claude Code sandboxing、IsolateGPT，用文件和网络 allowlist 限制本机权限。
- 沙箱抽象层：如 SWE-ReX、smolagents executor interface、Kubernetes Agent Sandbox，把不同后端包装成统一接口。

3.3 威胁模型与沙箱逃逸

agent 沙箱面对传统容器逃逸、资源耗尽和侧信道问题，也面对 agent 特有风险：prompt injection 可能诱导攻击操作，目标偏移可能让 agent 把逃逸当作工具性目标，多工具组合可能放大单点弱点。论文引用的 sandbox escape benchmark 显示，前沿模型在某些 Docker 配置下已有可观逃逸成功率，说明风险不是理论问题。

3.4 部署模式

当前有三种模式：自托管、云托管 SaaS、混合或 BYOC。自托管延迟低、控制强但运维负担大；云托管弹性好但有网络 and 合规成本；混合模式试图兼顾数据本地性与弹性执行容量。

3.5 小结

执行环境是 harness 的物理基座。它同时承担安全边界、评测重置机制和长期自主执行许可。不同方案的选择取决于威胁模型、任务真实性、成本、启动延迟和可移植性。

4 Tool Interface and Protocol Layer (T)

工具层决定 agent 如何发现能力、描述动作空间、调用外部能力并接收结果。它的基本张力是：工具越多，能力覆盖越广；但工具菜单越大，token 成本、选择错误、prompt injection 面和规划复杂度也越高。

4.1 协议和接口标准

MCP 是当前最显眼的工具/上下文集成协议，提供 host-client-server 结构和 JSON-RPC 交换，用于暴露 tools、resources 和 prompts。A2A 面向 agent 应用之间的通信与长程协作。OpenAI 风格 function calling 和 OpenAPI 仍是工具 schema 与 API 描述的重要基础。AGENTS.md/AGENT.md 则把 agent 工作约束写进仓库。

作者建议不要只按厂商比较协议，而要按照跨越的边界分类：Model 到 Function、Agent 到 Capability、Agent 到 Agent、Agent 到 Repo/Environment。

4.2 工具描述、发现和选择

关键问题不是“有多少工具”，而是“当前步骤该暴露哪些工具”。EASYTOOL、AnyTool、CRAFT、MetaTool、MCP-Zero、ToolRet、ToolRegistry 等工作都指向同一原则：少而高质量的工具常优于粗暴暴露全量工具。工具发现必须动态适应仓库、任务和租户环境。

4.3 工具增强训练和集成

Toolformer、Gorilla、ToolLLM/ToolBench、ToolkenGPT 等从模型训练或调用格式角度提升工具能力。生产系统还需要框架层 runtime，如 LangChain、Semantic Kernel、smolagents。对 coding agent 来说，工具还包括静态分析器、类型检查器、证明器、patch 等价检查器等。作者强调这些工具最好返回证据型产物，而不是简单 yes/no。

4.4 可扩展性与会话管理

长程任务会带来 stateful tool session：句柄可能过期，重试可能导致工具状态不一致，tool trace 可能挤爆上下文。因此 harness 需要显式生命周期控制、边界化的工具上下文注入，以及能区分 planner 错误和接口错误的 observability。

5 Context and Memory Management (C)

这一章讨论模型每一步“看见什么”，以及信息如何跨轮次、会话和任务持久化。

5.1 为什么必须工程化管理上下文

更大的上下文窗口并不自动解决记忆问题。原因包括：Transformer 注意力成本随 token 数近似二次增长；“Lost in the Middle”现象让中间位置的信息更难被使用；context rot 表明即使窗口没满，输入变长也会降低定位和推理质量。结论是：包含什么、放在哪里、何时移除，都会影响 agent 可靠性。

5.2 从 prompt engineering 到 context engineering

Prompt engineering 优化单次调用的静态输入；context engineering 优化多步任务中每一步的完整信息状态。生产 agent 的上下文包含系统提示、工具定义、历史轮次、工具结果、检索文档、记忆记录和动态工作状态。核心原则是：用最小的高信号 token 集合提升当前步骤成功概率。

5.3 短期：活动上下文窗口

短期上下文管理包括：

- 系统提示校准：从最小 prompt 开始，用失败模式驱动增量补充，避免预先堆满边界情况。
- token 高效工具设计：工具名、描述、参数本身消耗上下文；工具应少、清晰、自解释。
- 即时检索与渐进披露：先保留路径、查询、链接等索引，需要时再加载内容。
- KV-cache 友好设计：保持前缀稳定、上下文 append-only、序列化确定性，避免无意打破缓存。

5.4 中期：会话状态与跨运行持久化

中期技术介于当前窗口和长期记忆系统之间。常见做法包括结构化笔记、文件化计划、任务状态外部化、跨运行摘要注入。它们成本低、实用性强，但不适合大规模历史检索。

5.5 长期：持久记忆系统

长期记忆允许跨会话、跨任务、跨 agent 实例检索。MemGPT 把上下文窗口类比为 RAM，把外部存储类比为 disk。Generative Agents 提出 observation-reflection-retrieval 结构。MemoryBank 加入遗忘曲线和用户画像。

生产系统方面，Mem0 组合向量库、图数据库和 KV store；A-MEM 用类似 Zettelkasten 的动态知识网络；Hindsight 强调从经验中学习而不只是记住；Honcho 建立用户模型；Mozilla cq 探索跨 agent 的集体记忆。

5.6 长程技术：让 agent 在 100+ 轮后仍保持一致

长程任务需要组合短期、中期和长期机制。主要技术包括上下文压缩、工具结果清理、子 agent 上下文隔离、checkpoint/resume、自动触发 memory consolidation。重要原则是：上下文管理应该由 harness 承担，而不是完全交给模型自己记。

5.7 Context Drift 的限制

Context rot 是单次推理输入变长导致退化；context drift 是长程轨迹中 agent 逐渐偏离初始目标。压缩会丢信息，检索依赖正确 query，子 agent 隔离也不能解决 orchestrator 自身漂移。作者认为，仅靠 context engineering 不够，还需要评测、可观测、治理和人工检查形成完整 harness。

6 Lifecycle and Orchestration (L)

这一层关注任务如何跨模型调用、工具调用、失败、修订和交接推进。

6.1 生命周期状态管理

Lifecycle state 是 harness 用来继续执行的操作状态，如待办子任务、检查点、重试、共享 artifact、执行状态、仓库变更等。它不同于上下文记忆，也不同于 trace。无状态 replay 可复现但成本高；有状态执行连续性好但调试复杂；实践中常用混合模式。

6.2 单 agent 内循环

单 agent loop 延续 ReAct 的 observe-reason-act-observe 结构。Codex CLI、Claude Code、Gemini CLI、OpenCode、Aider、SWE-agent 等都可归入这一层。即使是单 agent，行为也由 prompt 构造、工具调用、反馈注入、状态管理和停止条件共同决定。

6.3 多 agent 编排模式

多 agent harness 将计划、执行、检查、聚合等职责拆给不同 agent。常见模式包括层级编排、团队编排、显式 workflow、fan-out 并行探索、图组合。代表系统包括 AutoGen、LangGraph、Semantic Kernel、OpenAI Agents SDK、DeepAgents、DeerFlow 等。好处是分工和并行，代价是协调状态与通信开销。

6.4 从 issue 到 pull request 的全生命周期

全生命周期 pipeline 把 agent loop 放进更大的软件工作流：issue/spec -> 计划 -> 实现 -> 测试 -> review -> PR。Vibe Kanban、Symphony、GitHub Agentic Workflows 等属于这一类。核心抽象是 task runner，它负责调度、状态持久化、重试、验证和迭代改进。

7 Observability and Operations (O)

作者把可观测性提升为独立层，因为生产 agent 需要专门的 trace、监控、成本、错误诊断和运维工具。

7.1 Trace 与监控平台

基础是结构化 trace：记录每次 LLM 调用、工具调用、检索、上下文组装为 span tree。Langfuse、Opik、Arize Phoenix、MLflow 等提供 trace tree、延迟、token、成本和 prompt versioning。OpenTelemetry 逐渐成为底层 instrumentation 标准，OpenLLMetry 和 OpenInference 将 LLM/agent 事件接入常规 observability 后端。

更深层的方案如 AgentSight 用 eBPF 从进程外监控 LLM 流量和系统行为；AgentTrace 设计 schema，把认知、操作、上下文三个 surface 纳入统一日志。

7.2 Agent 专用运维平台

AgentOps、RagaAI Catalyst、Laminar 等关注 session、agent 身份、角色、tool selection policy 和跨会话状态。学术上，AgentOps pipeline 被拆成 observe、collect metrics、detect anomalies、root cause analysis、recommend fixes、automate remediation。Cognitive observability 进一步追问 agent 为什么这么做，而不只是做了什么。

7.3 成本追踪和优化

agent 任务常由多次 LLM 调用构成，成本暴露会被放大。TensorZero、Helicone 等做成本追踪；FrugalGPT、GPTCache、QC-Opt、token-budget routing 等做路由、缓存和预算优化。论文强调，成本观测必须跨 API 调用、应用层路由和基础设施资源利用率。

7.4 可靠性工程

长程 agent 需要失败恢复、checkpoint/resume、重试和会话恢复。Anthropic 的实践将问题分解为任务拆解、进度 artifact、逐 feature 执行、干净交接状态。Managed Agents 架构把 brain、hands、session 分离，使 harness、sandbox 和事件日志都可替换和恢复。

学术工作提供故障分类，如多 agent 失败、memory/planning/action/system 模块错误、error propagation、version drift、成本驱动性能塌陷、认知退化。检测策略应分层：规则检查结构错误，统计监控 drift，LLM 分析语义/推理失败。

7.5 走向统一可观测

论文指出 observability 与 evaluation 常脱节：很多团队会收 trace，但不做离线 eval。理想状态是把生产失败 trace 自动转成回归测试，把在线评估分数变成告警信号，并通过可观测数据判断哪些 harness 组件仍然有用、哪些已成为成本和延迟负担。

8 Verification and Evaluation (V)

评测层把 agent 行为变成工程证据。作者主张，agent 分数应该被看作 model-harness pair 的性质，而不是模型单体性质。

8.1 Task-to-feedback 生命周期

论文把评测组织成五阶段质量控制循环：

1. 任务和 benchmark grounding。
2. 执行前 readiness validation。
3. 受控执行和 trace capture。
4. 多层判断与失败归因。
5. 持续回归和部署反馈。

8.2 阶段一：任务与 benchmark grounding

agent 任务不是一句自然语言 prompt，而是环境状态、可用工具、允许动作、约束、终止条件和成功标准的组合。软件工程任务如 SWE-bench、Terminal-Bench 用仓库状态和测试验证；WebArena、BrowserGym、OSWorld 等用浏览器/桌面状态定义成功；AgentBench、GAIA、The Agent Company 等覆盖更广泛跨域工作流。

8.3 阶段二：执行前 readiness validation

在 agent 运行前，必须确认沙箱、依赖、工具、上下文、权限、预算、grader 都初始化正确。否则失败可能来自环境、工具、权限或 grader，而不是模型。严谨评测应记录 tool registry、context policy、permission policy、budget、timeout、model config 和 runtime interface。

8.4 阶段三：受控执行与 trace capture

agent eval 的基本单位是 rollout：任务、模型配置、harness 配置、动作序列、中间观察、最终状态和评分。trace-native evaluation 要记录模型输出、工具调用、工具结果、状态变化、上下文快照、错误、重试、恢复动作、token、延迟和成本。

8.5 阶段四：多层判断与失败归因

Outcome-level evaluation 判断最终目标是否达成。Trajectory-level evaluation 判断路径是否合理、合规、高效。Evaluator-level evaluation 判断 grader 本身是否可靠。失败归因不是单标签分类，而是基于完整 trace 诊断模型、工具、上下文、沙箱、编排、benchmark 或 evaluator 哪一层出了问题。

8.6 阶段五：持续回归与部署反馈

harness 本身会持续变化：prompt、工具 schema、MCP server、context policy、sandbox image、permission rule、judge prompt 都可能改变。因此回归评测也应 by harness 变更触发，而不只是模型变更触发。生产 trace 应能转成 regression case，eval failure 应能转成 observability signal。

8.7 小结

评测不应只是 leaderboard。对长程 agent，更重要的问题是：为什么成功或失败，路径是否可接受，evaluator 是否可信，下一步应该改哪个 harness 组件。

9 Governance and Security (G)

这一层讨论如何限制 agent 行为、确保安全和可问责。由于 agent 能执行 shell、发邮件、提交代码、调用第三方 API，生产部署必须明确什么能做、谁授权、谁负责、如何审计。

9.1 权限模型与身份管理

静态权限边界如工作区文件权限、网络限制、命令 allow/deny list 容易检查但表达力有限。上下文相关 privilege control 使用策略语言或确定性 checker，在每次 tool call 前评估 tool name、参数和环境状态。多 agent 场景还需要身份、委托和短期访问 token，把人类主体、agent 实例和动作连接成可验证链条。

凭证管理也很关键：API key、session token、OTP 不应进入 LLM 上下文，常见做法是 vault 存储、上下文只暴露占位符、自动化层执行时替换真实值。跨网站/跨组织场景可能需要类似 robots.txt 的 agent-permissions manifest。

9.2 生命周期 hook

hook 决定治理检查在何时触发。论文将一个工具调用周期拆成四类 hook：

- H1：输入进入 LLM 前的 input guardrail。
- H2：工具执行前的 action validation/output guardrail。

- H3：工具输出回到上下文前的信息流控制。
- H4：高风险动作的人类确认。

PromptShield、DataSentinel 检测 prompt injection；ShieldAgent、ControlValve 检查动作；CaMeL 用 provenance tag 和信息流控制隔离不可信数据；Codex、Gemini CLI、Cursor、OpenHands 等对危险动作做人工批准。挑战是 hook API 不统一、延迟增加、多个 hook 之间可能相互干扰。

9.3 组件加固

模型加固通过 instruction hierarchy 或偏好优化，让模型区分高优先级系统指令和低优先级不可信输入。运行时分类器如 Llama Guard 可独立筛查输入输出。工具加固尤其关注 MCP 安全：有攻击研究显示恶意 MCP 工具描述可能影响模型，防御方向包括签名、版本化工具定义和协议级验证。

供应链风险也进入治理范围。模型可能幻觉出不存在的包名，攻击者注册这些包名后植入恶意代码。治理不能只覆盖 tool call，还要覆盖包、数据源和检索来源。

9.4 声明式 constitution

把治理逻辑写进业务代码会难以审计。新趋势是把规则外部化到 YAML 或 DSL，形成可读、可版本化、可 diff 的 agent constitution。训练时 constitution 如 Anthropic Constitutional AI 影响模型默认行为；部署时 YAML constitution 可被 harness 读取和执行；更强的 DSL 或形式化规范能表达顺序约束、权限升级、批准门和 UI 状态转移。

开放问题是：没有通用 schema，规则可移植性差；如何验证 constitution 无冲突且覆盖完整；训练时对齐与部署时规则冲突时谁优先。

9.5 审计基础设施

审计记录 agent 做了什么、为什么做、策略是否被遵守。理想审计记录至少包含 trace id、主体身份、工具调用、策略决策和版本、执行结果、资源成本、输入输出 hash。检测可以是 per-action，也可以是 trajectory-level。前者便宜可内联，后者能发现多步攻击但延迟高、定位难。

治理还包括成本和资源审计。多 agent fan-out 可能指数级放大调用成本，因此 session budget、token limit 和异常消费检测都属于治理。

9.6 放在 agent 安全图景中

论文把治理机制映射到 agent 安全风险：不可信接口、错误指令服从、无约束数据流、幻觉、数据泄露、未授权动作、资源耗尽。现实系统通常只部分实现防御，尤其缺少信息流控制、身份管理、形式化验证和自动异常检测。

9.7 研究方向

治理层的开放方向包括：标准化 policy 和 audit 语言、形式化治理保证、自适应治理、长程 agent 的会话级治理、可用的权限和审计 UI、跨层治理一致性、端到端供应链治理、统一 adversarial benchmark。

10 Cross-Cutting Concerns

第 10 章指出七层并非独立 checklist。执行环境会限制可用编排策略；上下文策略影响评测复现；治理跨越所有层；标准化协议要求 provenance、权限、成本和失败证据也能跨系统保留。

论文列出跨层常见缺口：工具互操作、成本归因、失败恢复、多仓库编排、人-agent 交接。这些问题说明，关键不是每层有没有工具，而是组合后的 harness 是否像可靠控制系统一样工作。

11 Cross-Layer Synthesis

11.1 成本-质量-速度三难

更强沙箱、更丰富上下文、更深评测、更完整 trace 都能提升质量和安全，但会增加 token、延迟、基础设施、存储和标注成本。生产系统必须决定哪些检查同步执行，哪些放到离线回归，哪些风险值得昂贵控制。

11.2 能力-控制权衡

工具越多、内存越持久、沙箱越开放，agent 覆盖任务越广，但误用和攻击半径也越大。能力与控制不是两个独立问题，而是一条贯穿工具 schema、上下文策略、运行权限、身份、审计和人工批准的设计轴。

11.3 Harness 耦合问题

局部优化可能在系统层变坏。工具描述会消耗上下文并改变模型行为；执行环境会改变评测结果；trace 如果没有身份和权限状态，就不能成为治理证据；评测设计会奖励或惩罚某些恢复循环。因此 harness 变更应作为系统变更测试。

11.4 从 agent framework 到 agent platform

生态正在从 framework 走向 platform。Framework 提供 agent、tools、memory、loop 等本地抽象；platform 增加持久 workspace、托管沙箱、身份、计费、可观测、评测、治理和人类交接。问题从“如何构建一个 agent”变成“如何运营一组可审计、可恢复、可逆的 agent”。

11.5 开放研究议程

研究需要把 harness 看作自适应控制系统：benchmark 应显式改变 harness 干预，而不只换模型；trace 应成为失败归因对象；不同 agent、工具、沙箱、评测器和人类之间需要标准化状态和责任交接；随着模型变强，harness 还应自动简化。

12 Open Problems and Future Directions

12.1 加固并扩展执行环境

需要统一的 agent-native runtime security 评测，覆盖 prompt injection、目标偏移和组合放大；需要成本模型来决定容器、microVM、OS 权限边界、桌面 VM、浏览器环境或学习型 surrogate 何时合适；还需要在自托管、云和混合部署之间保持语义一致的可移植层。

12.2 维护长程 agent 的可靠状态

上下文管理应被重新理解为状态估计问题。未来系统需要不确定性感知摘要、记忆 provenance、矛盾处理、显式过期标记，以及从持久 artifact 重建状态的能力。记忆策略不应只评测 recall，还要看是否减少后续动作错误。

12.3 从 agent trace 诊断失败

最终分数不足以解释失败。trace 应成为计算 outcome、trajectory quality、failure attribution 和 regression test 的主对象。当前 observability 与 eval 脱节，未来应把异常生产 trace 自动转成回归用例，并把诊断信号反馈到 prompt、tool、context 和 orchestration。

12.4 标准化 agent、工具和人的交接

交接不应只传一段文本摘要，还应传 intent、constraints、permissions、artifacts、provenance、budget state、risk level、trace history 和 unresolved decisions。关键是协议足够丰富以支持安全和恢复，又足够简单以被广泛采用。

12.5 模型变强后如何保持 harness 有用

每个 wrapper、reset、verifier、planner、memory rule、permission gate 都隐含“模型自己还做不好”的假设。模型升级后，这些假设可能失效。未来 harness 需要 shadow mode、A/B test、模块消融和自动搜索，在质量、延迟、成本和风险之间持续简化自己。

13 Conclusion

论文结论是：agent harness 是独立工程面，现实可靠性的上限不只由模型决定。ETCLOVG 把生产系统中已经分化出的职责形式化；大规模项目映射说明生态已经有丰富实践，但可观测和治理仍偏薄；从 prompt 到 context 再到 harness 的演进表明，agent 工程的核心对象正在从单次模型调用转向长程控制系统。

作者也承认限制：语料偏向英文、GitHub 可见、开源和 coding agent；ETCLOVG 目前主要是描述性框架，未来还需要发展成能指导设计决策的规范性框架。

图表中文说明

Figure 1

中文标题：Prompt engineering、Context engineering 与 Harness engineering 的简要比较。

含义：这张图用于说明三种工程对象的扩大：从优化提示词，到优化每步上下文，再到优化完整 agent 运行底座。

Figure 2

中文标题：LLM agent harness engineering 分类示意图。

图中中文标签建议：

- Structural pillars：结构支柱。
- Execution / Tooling / Context / Lifecycle：执行、工具、上下文、生命周期。
- Observability：可观测。
- Verification：验证与评测。
- Governance：治理与安全。

Figure 3

中文标题：2022-2026 年代表性 agent harness 系统时间线。

含义：从早期单 loop agent，演进到覆盖执行环境、工具协议、上下文记忆、编排、可观测、验证和治理的复杂底座。

Figure 4

中文标题：Agent harness engineering 分类树。

中文化要点：图中每个分支对应 ETCLOVG 一层及其子类。例如 E 层包括通用托管沙箱、computer-use 基础设施、代码沙箱、框架内置 runtime、浏览器环境、OS 权限沙箱和沙箱抽象层。

Figure 5

中文标题：项目语料构建流程。

含义：候选项目来自 GitHub、论文、curated list、包注册表和公司工程资料，经过去重、纳入标准检查后映射到 ETCLOVG 层。

Figure 6

中文标题：执行环境与沙箱代表工作。

含义：把 Daytona、E2B、Modal、Anthropic Computer Use、OpenHands、WebArena、IsolateGPT、SWE-ReX 等按沙箱类别归档。

Figure 7

中文标题：工具接口与协议代表工作。

含义：展示 MCP、A2A、function calling、OpenAPI、工具检索、工具增强训练、session management 等方向。

Figure 8

中文标题：上下文与记忆管理代表工作。

含义：按短期活动窗口、中期会话状态、长期持久记忆、长程上下文技术和 context drift 分类。

Figure 9

中文标题：生命周期与编排代表工作。

含义：分为单 agent loop、多 agent orchestration、从 issue 到 pull request 的全生命周期 pipeline。

Figure 10

中文标题：可观测与运维代表工作。

含义：覆盖 trace/monitoring 平台、agent-specific ops、成本追踪优化、可靠性工程和统一可观测。

Figure 11

中文标题：验证与评测代表工作。

含义：按照 task-to-feedback 生命周期组织 benchmark、readiness validation、controlled execution、multi-level judgement 和 continuous regression。

Figure 12

中文标题：Harness evaluation 的 task-to-feedback 生命周期。

中文化阶段：1. 任务和 benchmark grounding。2. 执行前 readiness validation。3. 受控执行与 trace capture。4. 多层判断与失败归因。5. 持续回归与部署反馈。

Figure 13

中文标题：治理与安全代表工作。

含义：覆盖权限与身份、生命周期 hook、组件加固、声明式 constitution、审计基础设施和 agent 安全图景。

Figure 14

中文标题：一个工具调用周期中的四个治理 hook。

中文化标签：

- Input：输入。
- LLM：语言模型。
- Tool：工具。
- Response：响应。
- InputGR：输入护栏。
- ActionGR：动作护栏。
- Post-exec IFC：执行后信息流控制。
- Human-in-the-Loop：人在回路中确认。

表格中文说明

Table 1

中文标题：工具/接口标准按集成边界和 harness 能力轴分类。

核心含义：Function calling、MCP、OpenAPI、A2A、ACP/ANP、AGENTS.md/AGENT.md 解决的是不同边界问题，不应简单互相替代。

Table 2

中文标题：代表性编排系统。

核心含义：系统按单 agent、多 agent、全生命周期 pipeline 分类，并比较主要编排模式和执行模型，如 stateless replay、stateful、hybrid。

Table 3

中文标题：治理机制与 agent 风险类型映射。

核心含义：权限、输入/输出护栏、信息流控制、组件加固、监控审计、人类确认、权限分离、形式化验证分别缓解不同风险。

Table 4

中文标题：治理功能覆盖情况。

核心含义：现实系统对权限、hook、加固、constitution、审计、多 agent 治理的支持很不均衡，很多能力只是部分支持或缺失。

术语表

- Agent harness：把模型调用变成长程、可控、可观察、可恢复 agent 执行的基础设施层。
- Binding constraint thesis：长程 agent 的可靠性瓶颈常来自 harness，而非模型单体。
- ETCLOVG：Execution, Tooling, Context, Lifecycle, Observability, Verification, Governance。
- Context rot：单次长上下文输入中，模型随输入增长出现定位和推理退化。
- Context drift：多轮长程执行中，agent 的内部状态逐渐偏离真实任务状态或初始意图。
- Trace-native evaluation：把完整执行 trace 作为评测和失败归因的主对象。
- Capability-control tradeoff：暴露更多能力会扩大可完成任务范围，也扩大误用和攻击半径。
- Harness coupling problem：harness 各层高度耦合，局部改进可能导致系统级退化。